

How virtual actors will help you scale your applications the easy way

11.05.2022

In the past, many applications were developed without the need to scale to large volumes of data, this made the programming relatively easy. It was usually enough to apply a simple object oriented model and crunch the numbers in memory. However, in modern times, scaling and processing large volumes of data, especially in IoT solutions,

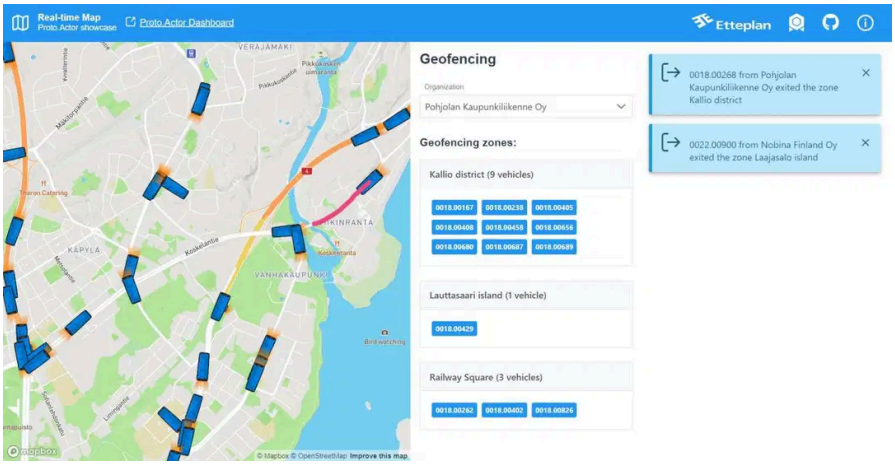
In the past, many applications were developed without the need to scale to large volumes of data, this made the

memory. However, in modern times, scaling and processing large volumes of data, especially in IoT solutions, are key factors of application's performance and strategic advantage. Scaling is the real problem and it brings the whole new level of complexity with it. The tech stack becomes unmanageable for small teams of developers.

However, there might be a way to scale to multiple threads and multiple machines while still keeping the easy programming model of handling of real-time events. All this while working in the comfort of the preferred and familiar programming platform. Let me introduce you to virtual actors model.

The problem

Let’s start with a little demo. Take a look at a sample application built as a showcase for the Proto.Actor framework. It tracks the buses in Helsinki in real time.



Let’s assume for a moment that we would like to build a similar application, and we have following requirements:

- Show the latest position of the bus.
- Show its trail over the last 5 minutes.
- Update the map in real time, but only the visible part. Limit the bus positions transferred over the network to only those that the user cares about.

- When the bus is standing still for more than 10 minutes and the door is closed, send a notification to the user.
- An organization that owns the bus can define geofences. Notify the user if a bus has entered or exited the geofence.

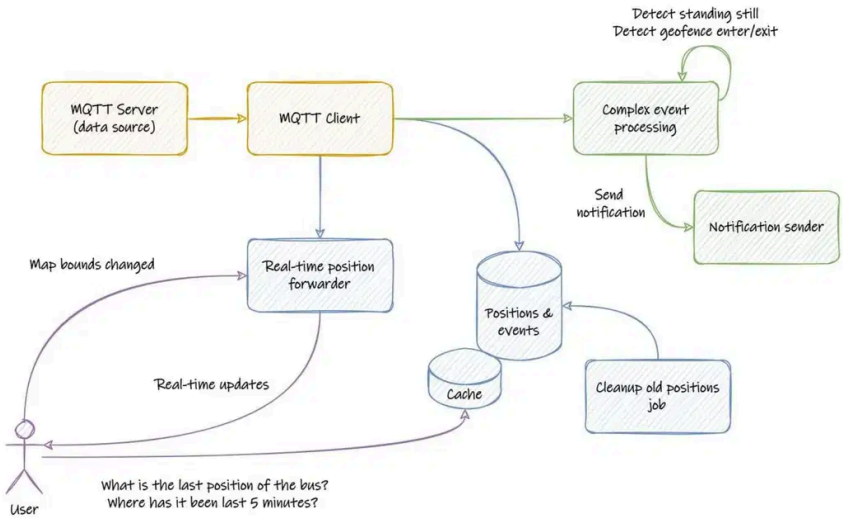
You could argue that Helsinki with its 1000+ public transport vehicles active at a time is not really a large scale application. So let's also say that:

- The application should scale to all major cities in Europe.

How do you design an application like that?

Traditional approach

There are various ways to solve this problem, and here's one of them:



There are a lot of problems we face with this approach:

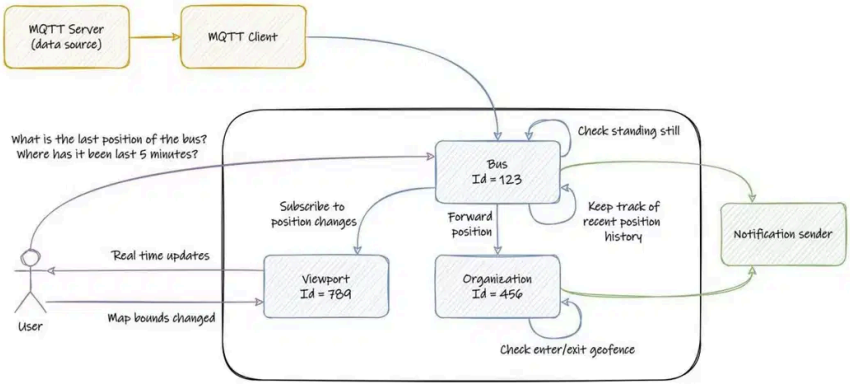
- We do not have a model of the real world in this application (no buses, no organizations). Instead we're thinking in terms of a "stream of positions" that needs to be processed.
- We have expensive queries on the positions database, that need to be cached.

communicating over the network.

- We have concurrency issues to deal with.
- We need a robust tech stack, which also means we need people who have broad competences to work with this stack.
- Overall, this architecture seems more complex than required for a relatively simple problem. The complexity is introduced by the scaling issues.

Can we do this in a simpler way?

It would be great if we could just work with a single application, represent the real-world concepts as objects and run all required algorithms in memory.



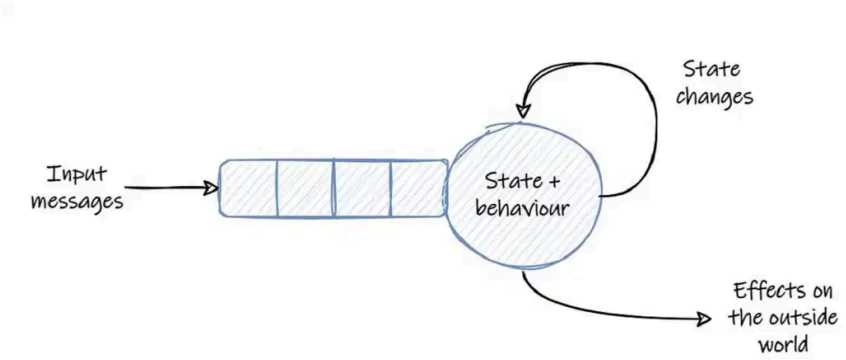
Here we represent each bus, organization, and viewport as unique objects and model interactions between them. Each object has a state and ability to mutate it, reacting to signals from the outside world and other objects. This application is much easier to build than the previous one, and definitely much easier to comprehend.

At the same time, you could say that simple object oriented app will not scale according to our requirements. We will not be able to fit it on a single machine and there is no way to scale it out.

of the real world while allowing to scale to large clusters of machines, each supporting a subset of your objects.

The actor model

Let's first talk about regular actors and the original actor model that was introduced in 1973. It is defined as a "mathematical model of concurrent computation", but let's explain that in simpler terms.



Think about the actor as an object that processes messages from a queue. It also keeps some record of internal state in its memory, e.g. recent history of positions of a bus (bus actor) or list of buses currently in a geofencing area (geofence actor). Each message might change the in-memory state which results in super-fast processing with no database involved. Some actors can also have an effect on the outside world.

Messages are processed one by one on a single thread. The state is private, so we have no concurrency issues and the code can be quite simple and easily testable.

However, given the single threaded nature of an actor, does this solution scale? The answer is yes, and you can achieve massive parallelism by having a large number of actors operating in parallel and communicating with messages. E.g. you can have one actor for each bus in Helsinki.

If all of this sounds too abstract to you, let's take a look at some code.

```
public class HelloActor : IActor
{
    public Task ReceiveAsync(IContext context)
    {
        if (context.Message is Hello helloMsg)
            Console.WriteLine($"Hello {helloMsg.Who}");

        return Task.CompletedTask;
    }
}
```

This simple actor processes Hello messages and prints output to the console (effect on the outside world).

```
public class BusActor : IActor
{
    List<Position> _positionHistory = new();

    public Task ReceiveAsync(IContext context)
    {
        switch (context.Message)
        {
            case Position position:
                _positionHistory = _positionHistory
                    .Where(p => DateTimeOffset.UtcNow - p.Timestamp < TimeSpan.FromMinutes(5))
                    .ToList();

                _positionHistory.Add(position);
                break;

            case GetPositionsHistoryRequest:
                context.Respond(new PositionBatch { Positions = { _positionHistory }});
                break;
        }

        return Task.CompletedTask;
    }
}
```

The *BusActor* represents a single bus in Helsinki. Each bus keeps a record of internal state - a list of recent positions. Each new Position message can mutate the state. The current state can be also returned as a response to a request from the outside.

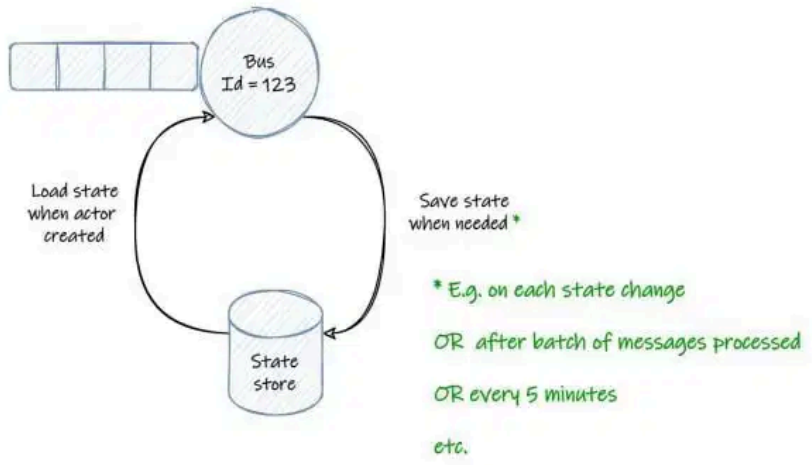
To give you more insight, these are some statements that characterize an actor:

- An island of consistency in a sea of concurrency
- Shared nothing
- Black box
- Location transparent
- Distributable by design

The practice shows that the logic implemented in the actor often takes form of a state machine. Because you are implementing actors in your favorite programming language with your preferred tools, this becomes an easy task. We think that this is a very powerful approach.

Persistence

In many cases keeping the state only in memory is not acceptable. It should survive machine restarts, application upgrades, etc. At the same time we previously said that processing is fast because it happens in memory. Is that a contradiction?



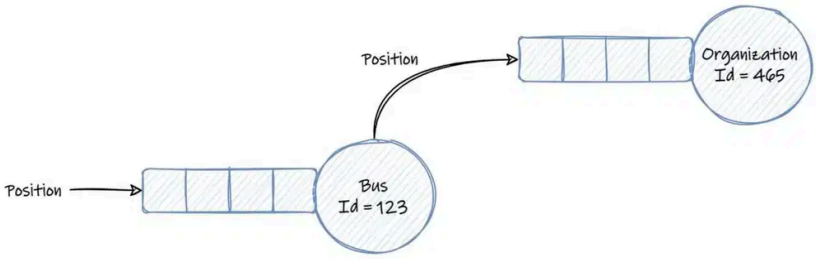
It is common to store the actor's state in some form of persistent storage, preferably a fast key-value store, like Redis, Azure Table Storage or Google Bigtable. However the way to think about this persistent state is more in terms of a snapshot or a backup, rather than regular operational data.

The state needs to be loaded only at the time when the actor is loaded into memory. We don't have to load state from the database before each message processing as would be the case with a traditional stateless application. This feature alone saves you 50% of the time normally spent doing IO. It is worth noting the in-memory state is a source of truth, not a simple cache - there are no cache invalidation problems.

state after each processed message, or after a number of messages are processed, or perhaps periodically e.g. every few minutes.

Communication

You can talk to an actor from the client or from another actor by passing a message to its queue. The communication is inherently asynchronous, although some of the frameworks make it easier by providing a *Task* (aka *Promise*) based APIs.



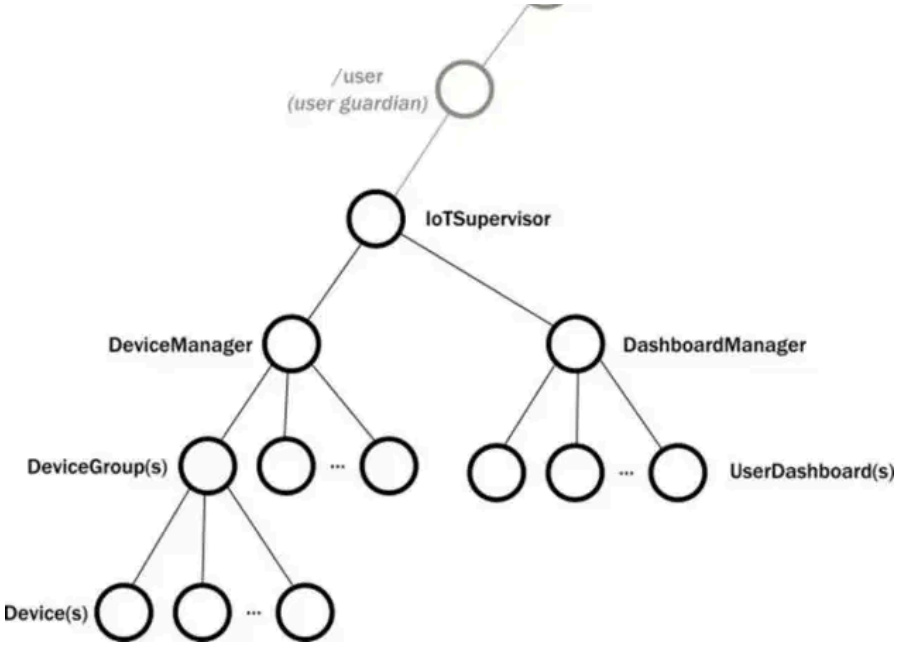
Potentially, this is how it would look in the code:

```
context.Send(organizationActorPid, position);
```

Notice here we are using a specific handle of the target organization actor in order to send a message to it. We will get rid of it in a moment.

Actor hierarchies

When you type “actor model” into your favorite web search engine, you will quickly stumble upon complex diagrams like this one:



Source: https://getakka.net/articles/intro/tutorial-1.html

In the traditional actor model, actors can form supervision hierarchies. This means that an actor can spawn child actors, it is responsible for their lifetime, and handles errors that appear in the hierarchy. In this approach:

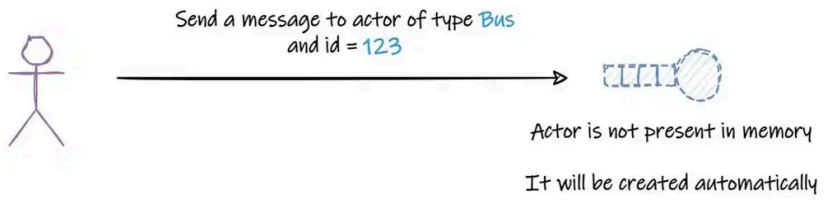
- There is an explicit lifecycle of the actor
- Errors are also handled explicitly, usually according to different strategies
- Communication between hierarchies hosted on different machines is not straightforward

Fortunately, in many applications such a complex approach is not really needed.

Let's talk about the virtual actor model.

Virtual actors

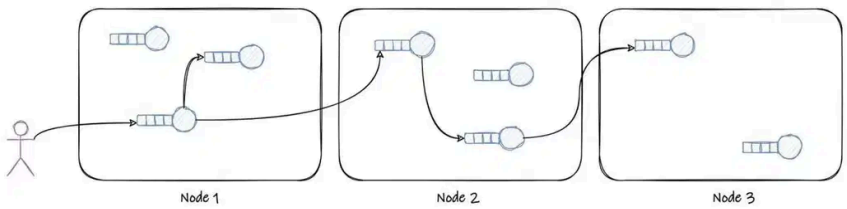
Virtual actors (or "grains") are assumed to always exist. You do not explicitly create nor destroy it. They might be currently active in the memory or be deactivated. However, from the perspective of a client that wants to communicate with it, it doesn't matter. You simply send a message and the virtual



What is also important is that instead of using some kind of handle or pointer to the actor, we specify the recipient of the message by type and id.

Clustering

These properties of a virtual actor make it possible to transparently scale out the model to multiple machines. The location of the actor is transparent, so we can talk to it as long as we know its type and id.



The framework will route the message to the actor, regardless of its current location. Also if the location changes (e.g. actor had to be migrated due to node shutdown), the client will still be able to reach it.

This approach gives you a lot of power and flexibility, while still maintaining a familiar and relatively simple programming model.

Tools

There are frameworks in the .NET space, that support virtual actor model, like Orleans, Proto.Actor, Akka.Net or parts of Dapr. We will look at them in more detail in the next post of the series.

As a rule of thumb, you should consider using actor model if your application deals with many small “entities” that each execute their logic independently and manipulate their own isolated state.

To be more concrete, we prepared a list of use cases that usually fit well with this approach:

- Modelling a device as an actor, with rules, alarm thresholds
- Digital twins
- Modelling user interactions as messages between actors
- Aggregating, map-reducing streams of data
- Tracking games, players, objects, resources, scores
- Matchmaking
- Distributing work among worker nodes
- Tasks that execute work and need to exchange information

Conclusion

The virtual actor model is a powerful approach to building applications. It enables building performant, distributed applications with tools and expertise that you already have. Stay tuned for next article in this series.

For more Information of if you have any questions, please contact:

Marcin Budny, R&D Lead

Marcin is a software architect and developer focused on distributed systems, cloud, APIs and event sourcing. He worked on projects in the field of IoT, telemetry, remote monitoring and management, system integration and computer vision with deep learning. Passionate about latest tech and good music.

Etteplan has offices in Finland, Sweden, the Netherlands, Germany, Poland, Denmark and China. Etteplan has also a sales office in the USA. We are a technology service company that, with our thinkers and doers, enable breakthroughs that help industrial businesses change.

Contact us →

→ References → Careers → Investors → Newsletter

Headquarters
Etteplan Oyj
Tekniikantie 4 D
02150 Espoo
Finland

Call center
+358 10 3070

General questions
info@etteplan.com

Financial administration
Askonkatu 9E
15100 Lahti
Finland

Services

- → Engineering
- → Etteplan rAIse – Pragmatic industrial AI for smarter businesses
- → Green Transition
- → Nearshoring and Offshoring services
- → Project Management Services
- → Software and Embedded Services
- → Technical Communication
- → Testing and Test Laboratory

Solutions

- → Aftermarket Solutions
- → Asset Development
- → Asset Operations and Maintenance
- → Order to Delivery
- → Production Development
- → Research & Development

Local sites

- → 中国
- → Danmark
- → Suomi
- → Deutschland

→ Facebook → Instagram → LinkedIn → YouTube

Privacy statement Privacy inquiry Accessibility Invoicing addresses

© Etteplan 2026